Taller Taxis NYC

Usar un clúster de AWS EMR con pyspark

1) Descargar el dataset de NYC - año 2018 taxis amarillos. Cargarlos en s3 en /taxis/viajes. Descargar archivo zones.csv y cargarlo en s3 /taxis/zonas.

   https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

2) Cargar viajes en un dataframe y zonas en otro.

3) Realizar un join para para realizar una consulta donde se liste: Lugar de recogida, hora_recogida, hora_dejada, DOLOCATION_ID(usar un alias para Lugar de recogida, PU_ZONE)

4) A partir de la anterior consulta realizar, una segunda consulta que cruce nuevamente y de el nombre de zona de dejada:
   PU_ZONE, DO_ZONE, PU_datetime, DO_datetime.

5) Crear una nueva columna trip_duration que sea la resta entre DO_datetime y PU_datetime. Para esto pasar a unixtimestamp las columnas. El resultado debe quedar en minutos.
   a) withColumn (crear columna)
      https://spark.apache.org/docs/3.1.3/api/python/reference/api/pyspark.sql.DataFrame.withColumn.html
   b) Pasar a unix_timestamp
      https://spark.apache.org/docs/3.1.3/api/python/reference/api/pyspark.sql.functions.unix_timestamp.html

6) Mostrar el Viaje que más ha durado
7) Determinar cuál es el destino más frecuente. Excluir posibles datos erróneos.
8) Determinar cuál es el viaje más frecuente(combinación origen-destino), y cuál el tiempo promedio de viaje. Incluir Desviación estándar.
9) Crear una columna de tipo entero que sea el año de cuando comenzó el viaje
10) Crear una columna de tipo entero que sea el mes de cuando comenzó el viaje
11) Listar los meses con número de viajes ordenados de mayor a menor
12) Guardar el dataset que incluye PU_ZONE, DO_ZONE, PU_datetime, DO_datetime, trip_duration, year, month en un bucket en S3 en formato parquet. Particionar los datos por year y month. La estructura debe quedar:
    s3://bucket/viajes/year=xxxx/month=xxxx
13) Ejecutar un crawler en Glue que cree una tabla "viajes" en una BD llamada "taxis". Verificar que los datos se pueden consultar en AWS Athena
14) Convertir el notebook en un programa python-spark que se pueda ejecutar directamente en el cluster.